

Backend Architecture Design Document – Admitix

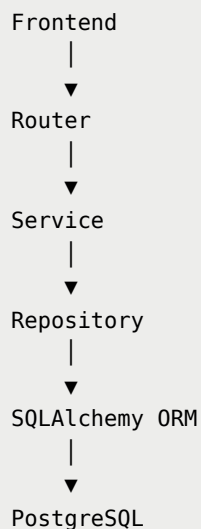
1. Project Overview

Purpose

Admitix is a backend-based College Admission Management System developed to automate and manage the admission process of educational institutions. The system provides REST APIs for managing students, admissions, documents, fees, seat allocation, notifications, and other admission-related activities.

The project follows a layered architecture where each layer has a dedicated responsibility. This separation improves code maintainability, readability, scalability, and simplifies future enhancements.

Backend Architecture



Objectives

- Build a modular backend architecture.
 - Keep business logic inside the Service layer.
 - Perform database operations through the Repository layer.
 - Reuse common utilities across modules.
 - Maintain clean, scalable, and maintainable code.
-

2. Project Folder Structure

The project is organized into common backend components and feature-based modules. Shared functionalities are placed inside the **common** folder, while feature-specific implementations reside in individual modules.

```
app/
├── core/
├── common/
│   ├── exceptions.py
│   ├── filters.py
│   ├── helpers.py
│   ├── pagination.py
│   ├── response.py
│   ├── utilities.py
│   └── validators.py
├── database/
├── modules/
│   ├── admission/
│   ├── student/
│   ├── payment/
│   ├── document/
│   └── ...
├── main.py
└── requirements.txt
```

Folder Responsibilities

Folder	Purpose
core	Stores application configuration, settings, and core components.
common	Contains reusable utilities, validators, pagination, responses, filters, and helper functions shared across all modules.
database	Manages database connection and SQLAlchemy configuration.
modules	Contains feature-wise implementation of backend modules.
main.py	Registers routers and starts the FastAPI application.

This organization separates shared functionalities from module-specific implementations, making the application easier to maintain and extend.

3. Module Structure

Every feature module follows the same standardized structure to ensure consistency throughout the project.

```

module/
|
├─ constants.py
├─ dependencies.py
├─ exceptions.py
├─ models.py
├─ permissions.py
├─ repository.py
├─ router.py
├─ schema.py
├─ service.py
├─ validators.py
└─ __init__.py

```

File Responsibilities

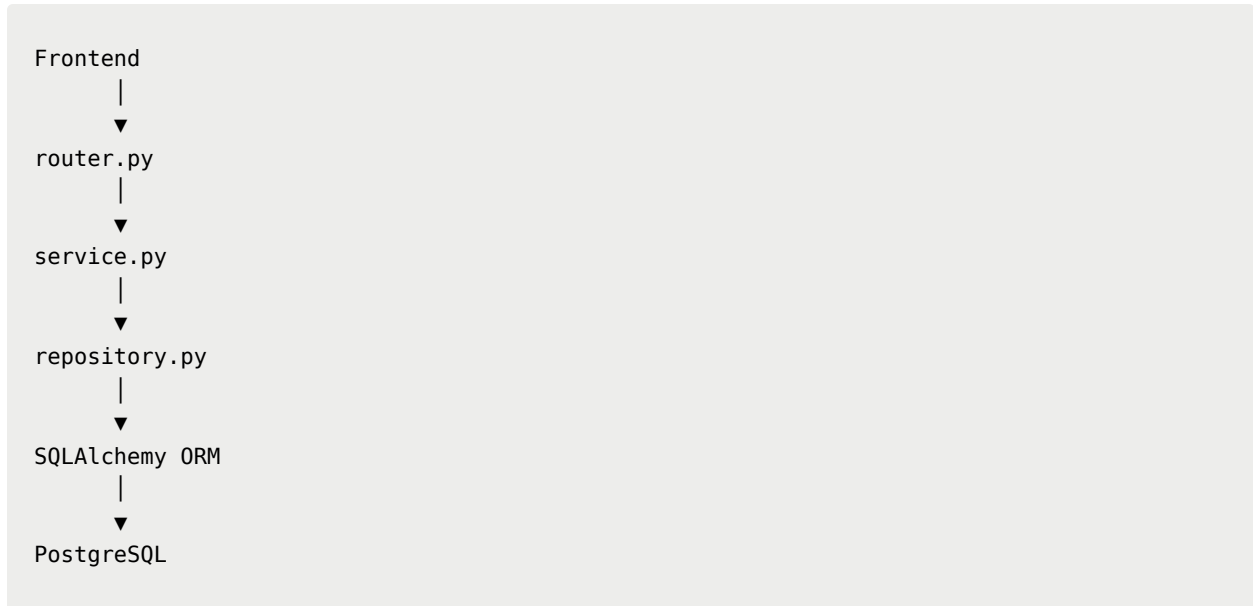
File	Responsibility
router.py	Defines API endpoints and forwards requests to the Service layer.
service.py	Implements business logic and coordinates application flow.
repository.py	Executes database operations using SQLAlchemy ORM.
models.py	Defines SQLAlchemy database models.

File	Responsibility
schema.py	Defines request and response schemas.
validators.py	Performs module-specific data validation.
permissions.py	Implements authorization and access control rules.
dependencies.py	Provides reusable dependencies for routers and services.
exceptions.py	Defines module-specific exception handling.
constants.py	Stores reusable constants used within the module.
init.py	Marks the directory as a Python package.

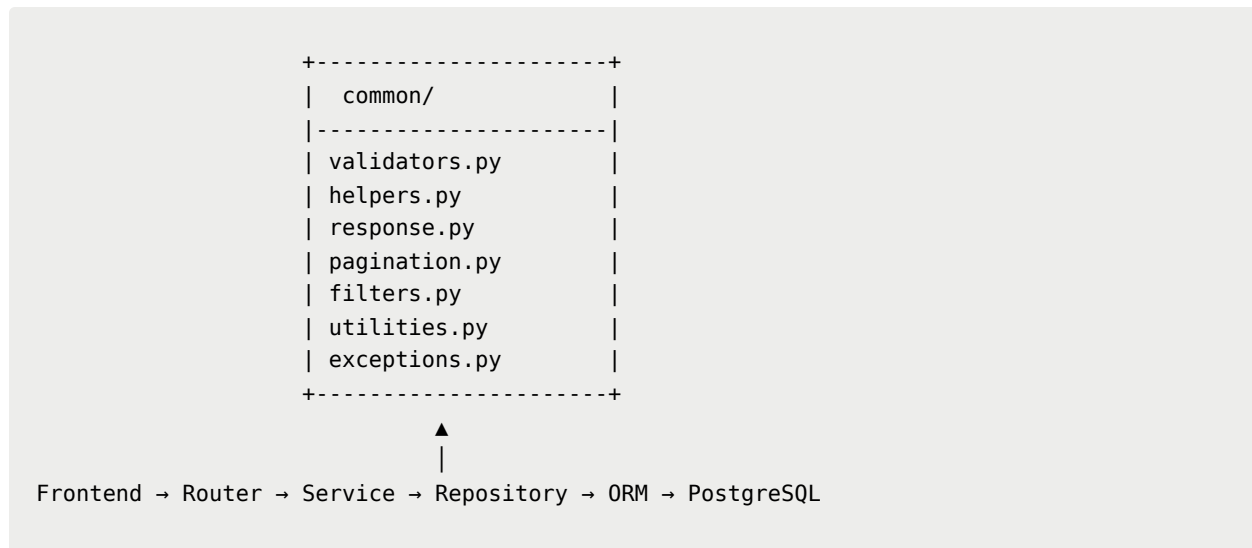
A uniform module structure simplifies development, debugging, and future maintenance.

4. Request Processing Architecture

Every request from the frontend passes through a sequence of backend layers before reaching the database.



During processing, the Service layer may use shared components from the **common** package and module-specific files as required.



Layer Responsibilities

Layer	Responsibility
Router	Receives HTTP requests and invokes service functions.
Service	Executes business logic and coordinates application flow.
Repository	Performs database operations through SQLAlchemy ORM.
ORM	Maps Python objects to database tables.
Database	Stores and retrieves application data.
Common Components	Provide reusable functionalities used across modules.

5. Router Layer (**router.py**)

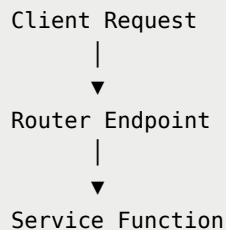
The Router layer exposes REST API endpoints for a module. It receives requests from the frontend, applies required dependencies or permissions, and delegates processing to the Service layer.

Responsibilities

- Define API routes.
- Receive request data.
- Apply dependencies.

- Apply permissions where required.
- Call service functions.
- Return standardized API responses.

Request Flow



Example Router Functions

HTTP Method	Service Function
POST	create_student()
GET	get_student()
GET	get_all_students()
PUT	update_student()
DELETE	delete_student()

The Router layer should remain lightweight and should not contain business logic or database operations.

6. Service Layer (**service.py**)

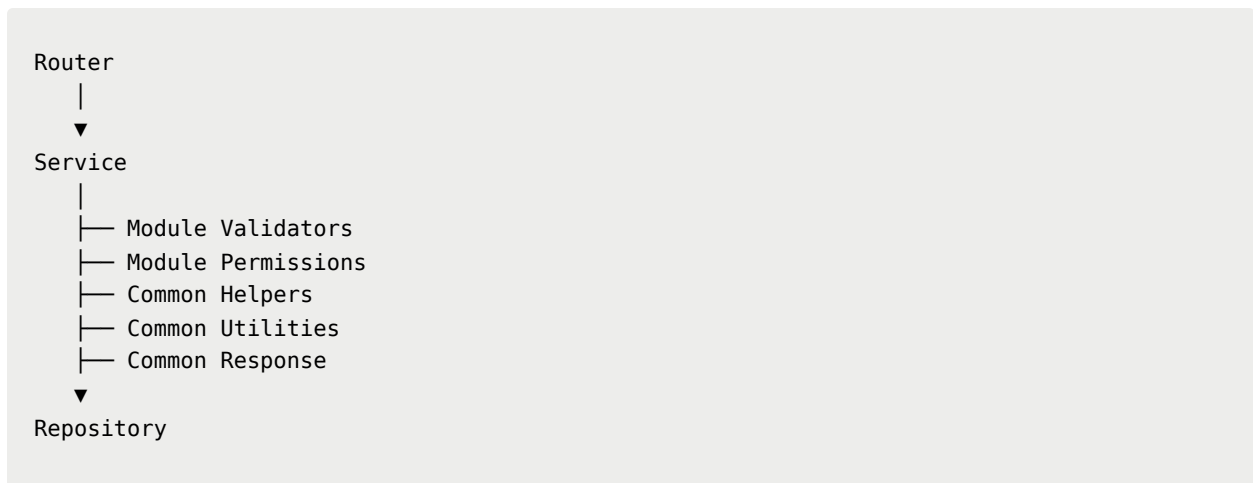
The Service layer is the core of the application where business logic is implemented. It validates business rules, coordinates operations, interacts with the Repository layer, and utilizes shared components from the **common** package whenever required.

Responsibilities

- Implement business rules.

- Coordinate application workflow.
- Invoke module validators and permissions.
- Use common helpers and utilities.
- Call repository functions.
- Prepare standardized responses.

Service Flow



Example Service Functions

Operation	Service Function
Create	create_student()
Retrieve	get_student()
List	get_all_students()
Update	update_student()
Delete	delete_student()

All business decisions and application logic should be implemented within `service.py`.

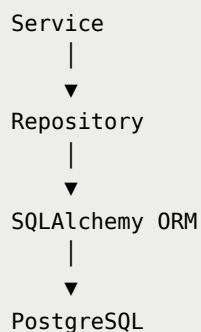
7. Repository Layer (`repository.py`)

The Repository layer is responsible for interacting with the database. It performs Create, Read, Update, and Delete operations using SQLAlchemy ORM and returns the required data to the Service layer.

Responsibilities

- Execute database queries.
- Insert records.
- Update records.
- Delete records.
- Retrieve data.
- Manage SQLAlchemy sessions.

Repository Flow



Example Repository Functions

Operation	Repository Function
Create	create_student_repo()
Retrieve	get_student_repo()
List	get_all_students_repo()
Update	update_student_repo()
Delete	delete_student_repo()

The Repository layer should contain only database-related operations and should not implement business rules.

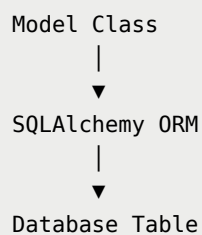
8. Database Models (`models.py`)

The `models.py` file defines the database structure using SQLAlchemy models. Each model represents a database table, including its columns, constraints, and relationships.

Responsibilities

- Define database tables.
- Specify columns and data types.
- Configure relationships between tables.
- Represent database entities as Python classes.

Model Flow



Model Components

Component	Purpose
Table Name	Represents a database table.
Columns	Define table attributes and data types.
Primary Key	Uniquely identifies each record.
Foreign Key	Establishes relationships between tables.
Relationships	Defines associations between related models.

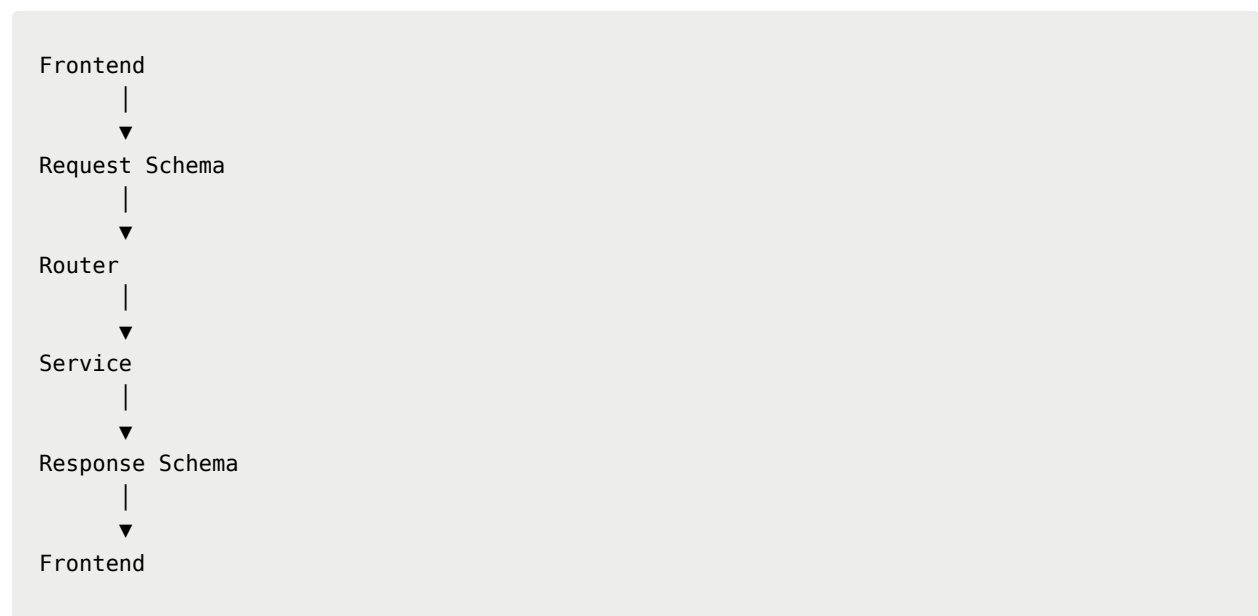
9. Schema Layer (`schema.py`)

The `schema.py` file defines the request and response structures used by the APIs. It ensures that incoming data and outgoing responses follow a consistent format.

Responsibilities

- Define request schemas.
- Define response schemas.
- Validate API request structure.
- Standardize API responses.

Schema Flow



Example Schemas

Schema	Purpose
StudentCreateSchema	Create a new student
StudentUpdateSchema	Update student details
StudentResponseSchema	Return student information
StudentListSchema	Return multiple student records

The Router receives validated request data through schemas and returns structured responses using response schemas.

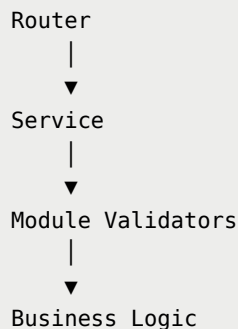
10. Module Validators (`validators.py`)

Each module contains its own `validators.py` file for validations that are specific to that module. These validations are executed before the business logic proceeds.

Responsibilities

- Validate module-specific input.
- Check required business conditions.
- Verify field values.
- Prevent invalid data processing.

Validation Flow



Example Validations

Validation	Purpose
Duplicate PRN	Prevent duplicate records
Admission Status	Check admission eligibility
Seat Availability	Verify available seats
Required Documents	Ensure mandatory documents are uploaded

Module validators focus only on validations related to their respective module.

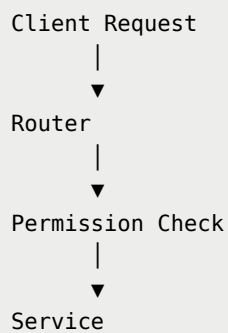
11. Permissions Layer (`permissions.py`)

The `permissions.py` file contains authorization rules that determine whether a user is allowed to perform a specific operation.

Responsibilities

- Verify user permissions.
- Restrict unauthorized actions.
- Control access based on user roles.
- Protect sensitive operations.

Permission Flow



Example Permissions

User Role	Allowed Operations
Super Admin	Full access
College Admin	Manage college admissions
Admission Officer	Verify applications
Student	Access own admission records

Permission checks should be performed before executing business logic.

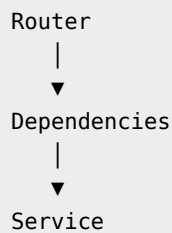
12. Dependencies Layer (`dependencies.py`)

The `dependencies.py` file contains reusable dependencies that can be injected into routers or services. This avoids repeating common logic across multiple endpoints.

Responsibilities

- Provide reusable dependencies.
- Supply database sessions.
- Retrieve authenticated users.
- Share common resources across APIs.

Dependency Flow



Example Dependencies

Dependency	Purpose
Database Session	Provides active database connection
Current User	Retrieves authenticated user
Current College	Retrieves current college information
Common Filters	Supplies reusable filter values

Dependencies reduce duplicate code and simplify endpoint implementation.

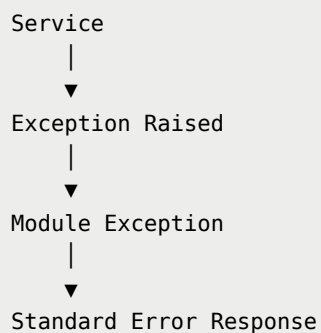
13. Module Exceptions (`exceptions.py`)

Each module contains its own `exceptions.py` file for handling errors specific to that module.

Responsibilities

- Define module-specific exceptions.
- Return meaningful error messages.
- Handle business rule violations.
- Maintain consistent error handling.

Exception Flow



Example Exceptions

Exception	Description
StudentNotFoundException	Student record not found
DuplicateApplicationException	Application already exists
InvalidAdmissionStatusException	Admission status is invalid
SeatUnavailableException	No seats available

Module exceptions improve readability and centralize error management.

14. Constants Layer (`constants.py`)

The `constants.py` file stores reusable constant values used throughout a module. This avoids hardcoding values in multiple places.

Responsibilities

- Store fixed values.
- Define status constants.
- Define message constants.
- Improve code consistency.

Example Constants

Constant	Purpose
ACTIVE	Active record status
INACTIVE	Inactive record status
PENDING	Pending approval
APPROVED	Approved status
REJECTED	Rejected status

Keeping constants in one location makes updates easier and improves maintainability.

15. Common Package

The `common` package contains reusable components that are shared across all modules. Instead of duplicating code, modules can import these components whenever required.

```
common/  
|  
├─ exceptions.py  
├─ filters.py  
├─ helpers.py  
├─ pagination.py  
├─ response.py  
├─ utilities.py  
└─ validators.py
```

Responsibilities of Common Files

File	Responsibility
exceptions.py	Defines reusable exception classes and common error handling.
filters.py	Provides reusable filtering logic for API queries.
helpers.py	Contains shared helper functions used by multiple modules.
pagination.py	Implements reusable pagination logic for list APIs.
response.py	Generates standardized API response formats.
utilities.py	Contains common utility functions such as formatting or conversions.
validators.py	Provides reusable validation functions shared across modules.

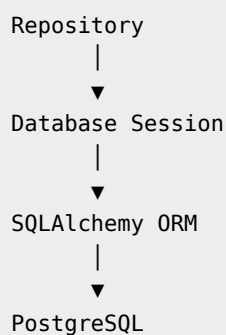
The `common` package promotes code reuse and keeps feature modules focused on their own business logic.

16. Database Connection and `main.py`

Database Connection

The database configuration is maintained in the `database` package. Repository functions use the configured SQLAlchemy session to interact with PostgreSQL.

Connection Flow



The database session is provided through dependencies, ensuring consistent and reusable database access across all modules.

`main.py`

The `main.py` file serves as the application's entry point. It creates the FastAPI application and registers the routers of all modules.

Responsibilities

- Create the FastAPI application.
- Register all module routers.
- Configure middleware.
- Initialize application settings.

Router Registration Flow

```
main.py
├── Student Router
├── Admission Router
├── Document Router
├── Payment Router
└── Other Module Routers
```

Application Startup

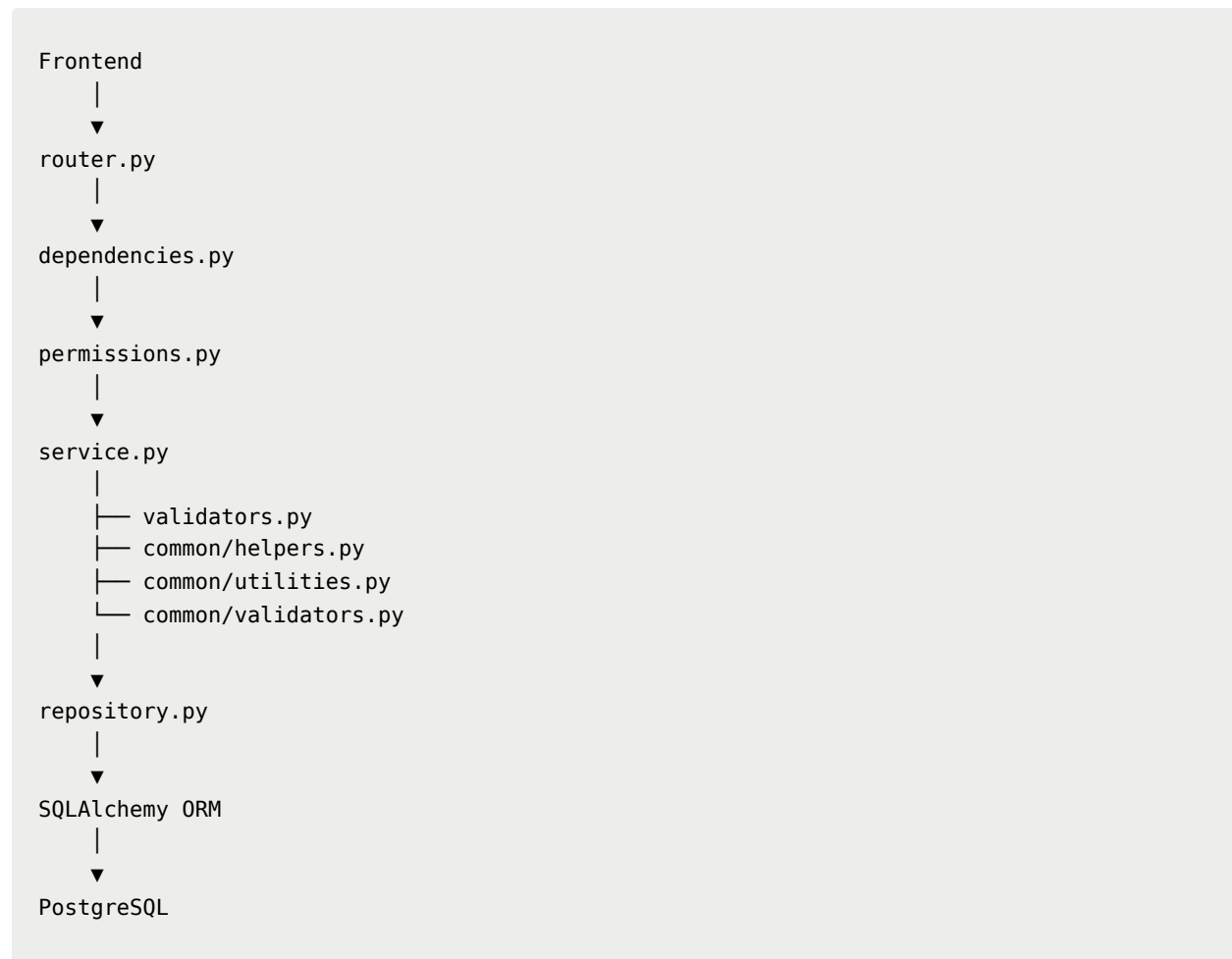
```
Application Start
┆
┆   ▼
┆ Create FastAPI App
┆   │
┆   ▼
┆ Register Routers
┆   │
┆   ▼
┆ Application Ready
```

`main.py` acts as the central entry point that connects all modules into a single backend application.

17. Complete Request Flow

Every request follows a fixed sequence from the frontend to the database. Each layer performs only its assigned responsibility.

Request Flow



Layer Responsibilities

Layer	Responsibility
Frontend	Sends API request
Router	Receives request and calls service
Dependencies	Provides reusable dependencies
Permissions	Verifies user authorization
Service	Executes business logic
Validators	Validates business rules

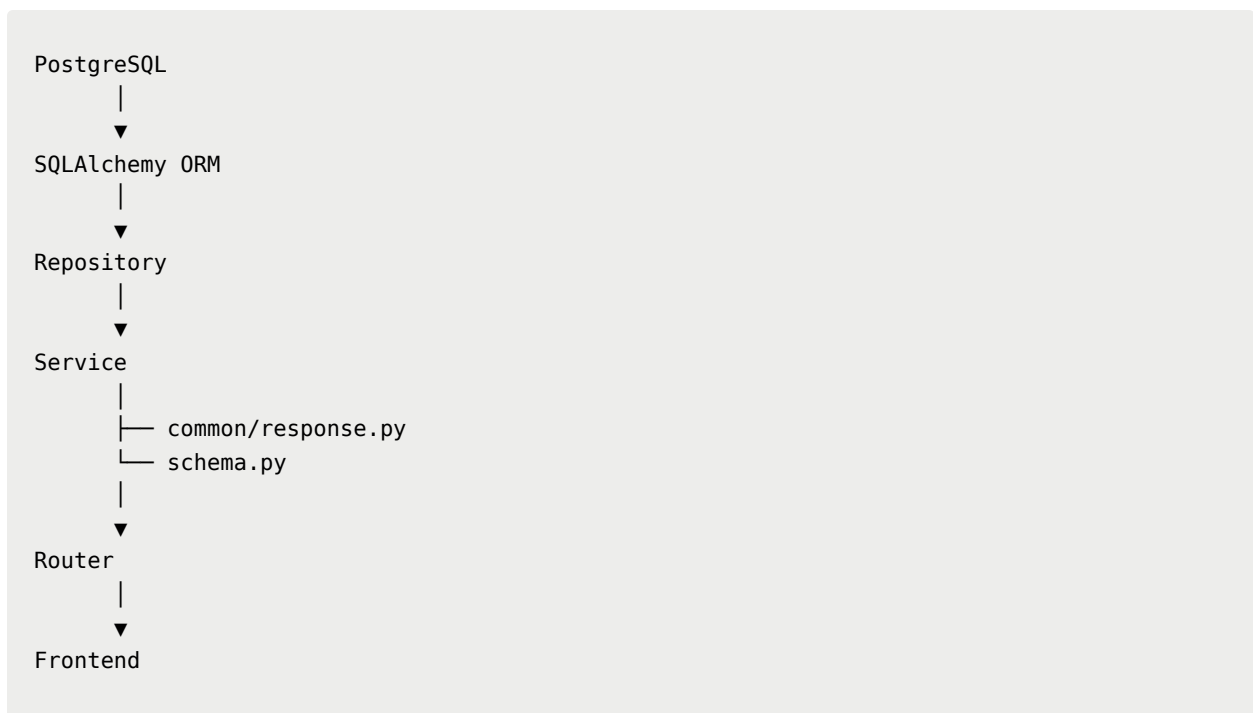
Layer	Responsibility
Repository	Executes database operations
ORM	Maps objects to database
PostgreSQL	Stores application data

Every request should follow this sequence to maintain a clean architecture.

18. Complete Response Flow

After completing the requested operation, the backend sends the response back through the same layers.

Response Flow



Response Processing

Layer	Responsibility
Repository	Returns database result
Service	Processes business result

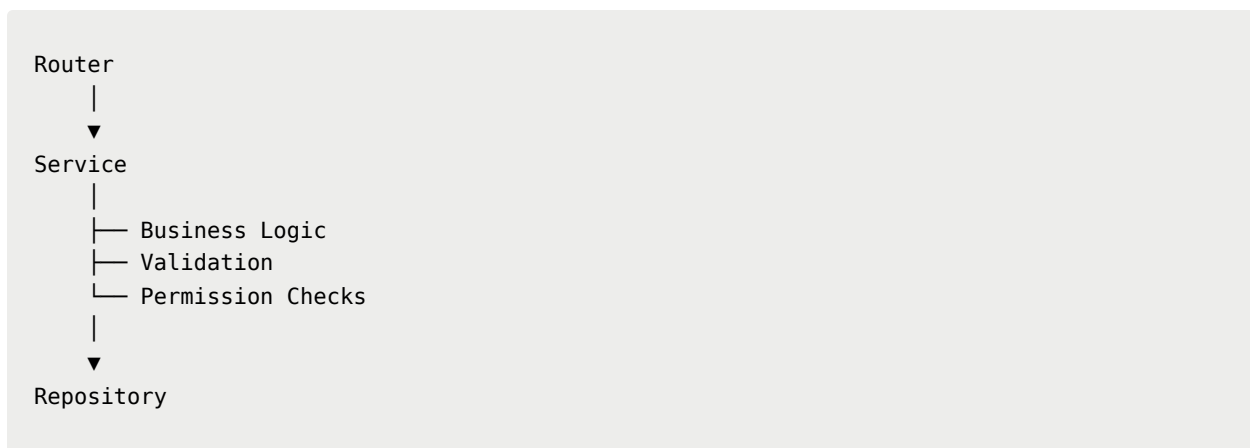
Layer	Responsibility
Response Utility	Creates standardized response
Schema	Formats response data
Router	Returns HTTP response
Frontend	Displays response

Responses should always follow a consistent structure across all APIs.

19. Service–Repository Communication

The Service layer does not directly interact with the database. All database operations are delegated to the Repository layer.

Communication Flow



Responsibilities

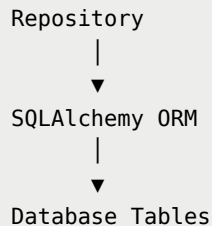
Service	Repository
Implements business rules	Executes SQL queries
Coordinates workflow	Retrieves records
Calls validators	Inserts records
Calls permissions	Updates records
Prepares responses	Deletes records

This separation allows business logic and database logic to evolve independently.

20. Repository–ORM Interaction

The Repository layer communicates with SQLAlchemy ORM instead of writing SQL directly.

Flow



Responsibilities

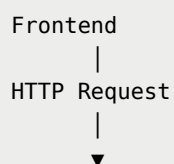
Repository Operation	ORM Action
Create	Insert row
Get	Select record
Get All	Select multiple records
Update	Update existing row
Delete	Remove record

The ORM converts Python model objects into database operations automatically.

21. Frontend Integration

The frontend communicates with the backend only through REST APIs exposed by the Router layer.

Integration Flow





Frontend Responsibilities

- Send API requests.
- Provide request data.
- Display API responses.
- Handle response status codes.

Backend Responsibilities

- Validate requests.
- Execute business logic.
- Perform database operations.
- Return structured responses.

The frontend should never interact directly with the database.

22. Standard API Response

All APIs should return responses in a consistent format. The `common/response.py` file should be used to generate standardized responses.

Response Structure

```
{
  success,
  message,
  data,
  errors
}
```

Example Response Types

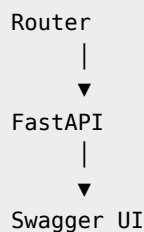
Response	Purpose
Success	Operation completed successfully
Error	Business rule violation
Validation Error	Invalid request data
Not Found	Requested resource unavailable
Unauthorized	Access denied

Using a common response structure simplifies frontend development and error handling.

23. API Documentation (Swagger UI)

Swagger UI provides interactive documentation for all APIs. As routers are registered in the application, their endpoints automatically become available in the documentation.

Swagger Flow



Uses

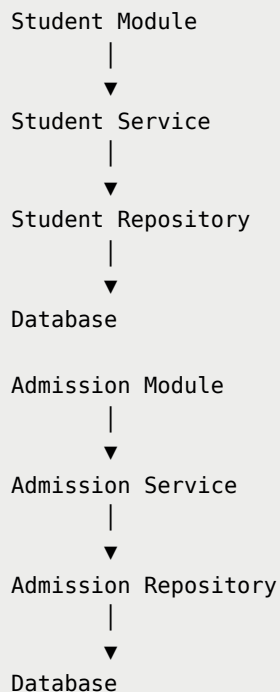
- View available APIs.
- Test API endpoints.
- Verify request schemas.
- Verify response schemas.
- Check endpoint parameters.

Swagger should always reflect the latest implementation of registered routers.

24. Module Interaction Workflow

Different modules interact through the Service layer while maintaining separation of responsibilities.

Workflow



If one module requires functionality from another, the interaction should occur through the appropriate Service layer rather than accessing another module's Repository directly.

Guidelines

Rule	Description
Module Independence	Each module manages its own business logic.
Repository Isolation	Repositories access only their own data.
Service Coordination	Services coordinate cross-module operations when required.

Rule	Description
Code Reuse	Shared functionality should use the <code>common</code> package.

This approach maintains modularity and reduces coupling between different parts of the application.

25. Coding Standards

The project follows a consistent coding style across all modules to improve readability, maintainability, and collaboration.

Standards

- Keep each layer responsible for only one task.
- Write meaningful function and variable names.
- Avoid duplicate code by using the `common` package.
- Keep functions small and focused.
- Use proper exception handling.
- Maintain consistent formatting across modules.

Layer-wise Responsibility

Layer	Responsibility
Router	API endpoints
Service	Business logic
Repository	Database operations
Models	Database tables
Schema	Request and response models
Validators	Data validation
Permissions	Authorization
Common	Shared reusable components

Following these standards keeps the codebase organized and easy to extend.

26. Naming Conventions

Consistent naming improves readability and makes the code easier to understand.

File Naming

File	Convention
router.py	API endpoints
service.py	Business logic
repository.py	Database operations
schema.py	Request/Response schemas
models.py	Database models
validators.py	Validation functions
permissions.py	Permission checks
dependencies.py	Reusable dependencies
constants.py	Constant values
exceptions.py	Custom exceptions

Function Naming

Layer	Example
Router	<code>create_student()</code>
Service	<code>create_student()</code>
Repository	<code>create_student_repo()</code>
Validator	<code>validate_student_data()</code>
Permission	<code>check_student_permission()</code>

Class Naming

Component	Convention
Model	<code>Student</code>
Schema	<code>StudentCreateSchema</code>

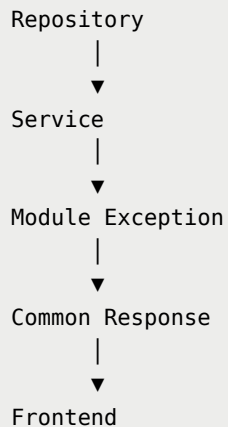
Component	Convention
Exception	<code>StudentNotFoundException</code>

Clear naming conventions make the project easier to navigate and maintain.

27. Error Handling Strategy

Errors should be handled consistently throughout the application to provide meaningful responses and simplify debugging.

Error Handling Flow



Strategy

- Detect errors as early as possible.
- Raise module-specific exceptions.
- Avoid exposing internal implementation details.
- Return standardized error responses.
- Log unexpected errors for debugging.

Example Error Types

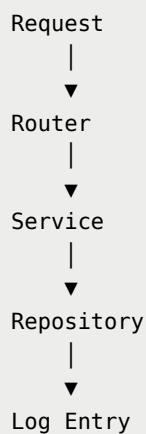
Error	Handling
Record Not Found	Raise module exception
Duplicate Data	Raise validation exception
Invalid Input	Return validation response
Unauthorized Access	Return permission error
Database Failure	Return internal server error

This approach ensures predictable and user-friendly error handling.

28. Logging Strategy

Logging helps monitor application behavior and simplifies debugging during development and production.

Logging Flow



Logging Guidelines

- Log important application events.
- Log database operation failures.
- Log unexpected exceptions.
- Avoid logging sensitive information.
- Use consistent log messages.

Common Logging Points

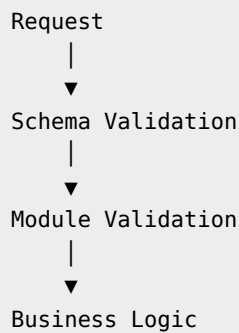
Event	Purpose
API Request	Track incoming requests
API Response	Monitor completed operations
Validation Failure	Identify invalid inputs
Database Error	Debug database issues
Exception	Troubleshoot application errors

Logging should support debugging without affecting application performance.

29. Validation Strategy

Validation ensures that only correct and complete data is processed by the application.

Validation Flow



Validation Levels

Level	Responsibility
Schema	Validate request structure
Module Validator	Validate business-specific rules
Service	Verify business conditions
Repository	Execute database operation only after validation

Examples

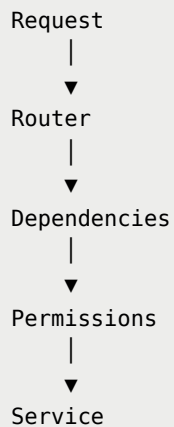
- Required fields
- Duplicate record checks
- Admission eligibility
- Seat availability
- Document completeness

Validation should occur before any database operation is performed.

30. Security and Permissions Workflow

Authorization checks ensure that users can access only the resources they are permitted to use.

Permission Flow



Workflow

1. Receive the request.
2. Identify the current user.
3. Determine the user's role.
4. Verify required permissions.

5. Continue to business logic if authorized.
6. Return an error response if access is denied.

Example Access Control

Role	Access
Super Admin	Full system access
College Admin	Manage college data
Admission Officer	Process admissions
Student	Access personal information

Permission checks should always be completed before executing business logic.

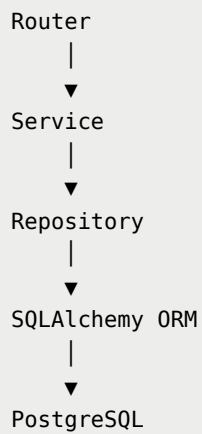
31. Project Best Practices

The following practices should be followed during development to maintain a clean and scalable codebase.

Development Practices

- Keep routers lightweight.
- Implement all business logic in `service.py`.
- Perform all database operations in `repository.py`.
- Reuse components from the `common` package.
- Use constants instead of hardcoded values.
- Raise custom exceptions where appropriate.
- Keep modules independent.
- Maintain consistent response formats.
- Write reusable validation and permission logic.
- Follow the established folder structure.

Recommended Architecture

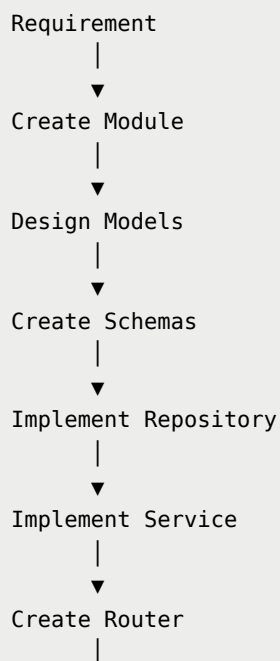


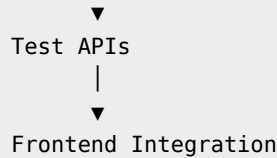
Following these practices ensures that the backend remains maintainable, modular, and easy to extend as new features are added.

32. Development Workflow

Development should follow a structured workflow to ensure consistency across all modules and simplify collaboration.

Development Steps





Workflow Description

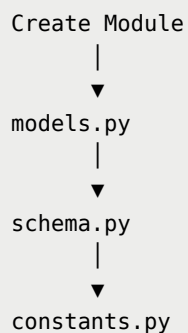
Step	Purpose
Requirement Analysis	Understand the feature requirements.
Model Design	Create SQLAlchemy models.
Schema Design	Define request and response schemas.
Repository Development	Implement database operations.
Service Development	Implement business logic.
Router Development	Expose REST API endpoints.
API Testing	Verify API functionality.
Frontend Integration	Connect APIs with the frontend.

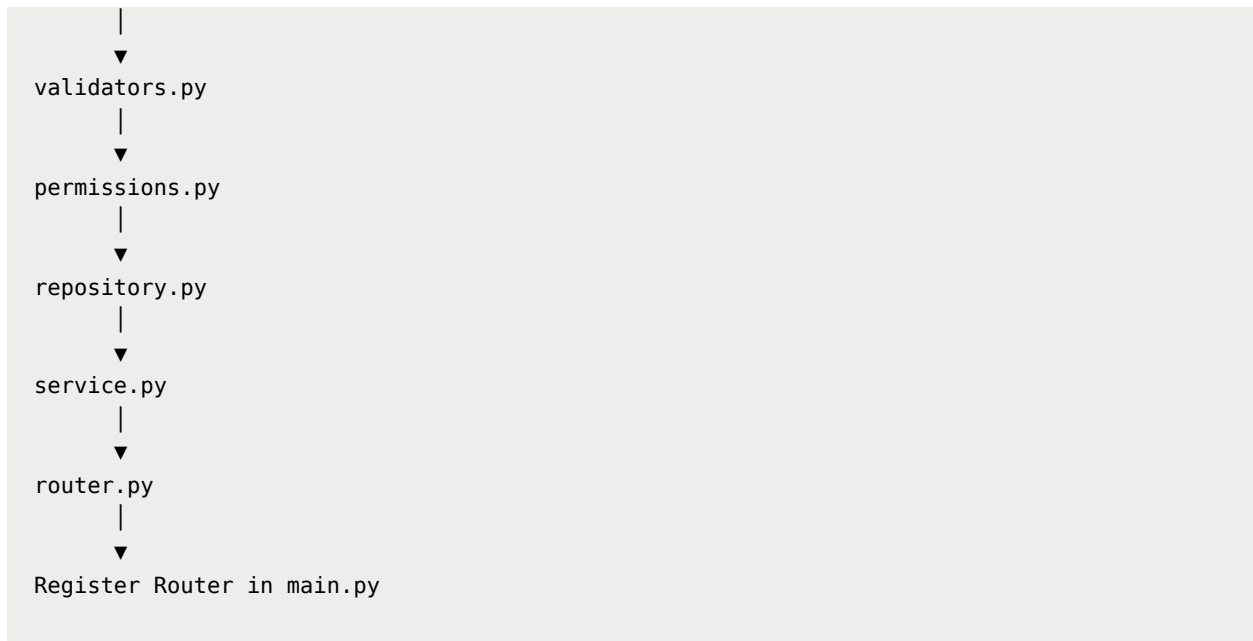
Following the same workflow for every module keeps development consistent and organized.

33. Module Implementation Workflow

Every module should follow the same implementation sequence to maintain a uniform project structure.

Implementation Flow





Implementation Responsibilities

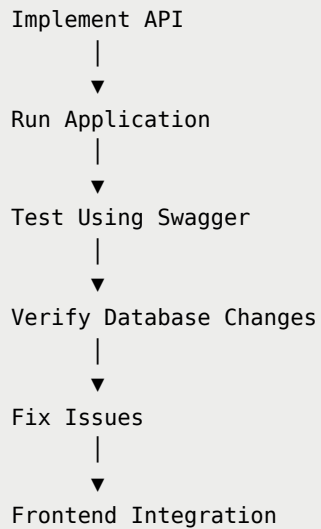
File	Responsibility
<code>models.py</code>	Define database tables.
<code>schema.py</code>	Define request and response models.
<code>constants.py</code>	Store reusable constants.
<code>validators.py</code>	Validate module-specific data.
<code>permissions.py</code>	Implement access control.
<code>repository.py</code>	Handle database operations.
<code>service.py</code>	Implement business logic.
<code>router.py</code>	Expose API endpoints.
<code>main.py</code>	Register the router with the application.

This implementation sequence ensures every module follows the same architecture and development standards.

34. API Testing Workflow

After implementing a module, its APIs should be tested before integrating them with the frontend.

Testing Flow



Testing Checklist

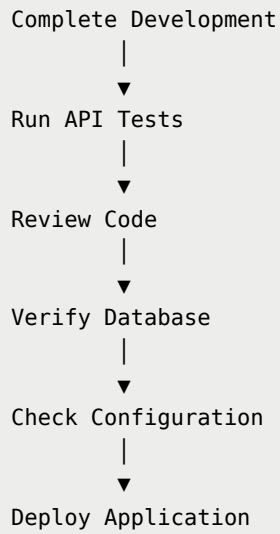
Check	Description
Request Validation	Verify request schemas.
Response Validation	Verify response format.
Business Logic	Confirm expected behavior.
Database Operation	Verify data persistence.
Exception Handling	Test error scenarios.
Permission Check	Verify access control.

Testing each API before frontend integration reduces debugging effort later.

35. Deployment Preparation

Before deployment, the backend should be reviewed to ensure that it is stable, secure, and ready for production.

Deployment Checklist



Pre-Deployment Checklist

Item	Status
All routers registered	✓
Models created	✓
Schemas completed	✓
Repository implemented	✓
Service implemented	✓
Validations completed	✓
Permissions configured	✓
Exception handling verified	✓
API responses standardized	✓
Swagger documentation verified	✓

Completing this checklist helps ensure a smooth deployment process.

36. Project Summary

Admitix follows a layered architecture where each layer has a specific responsibility. This design improves modularity, readability, maintainability, and

scalability.

Overall Architecture

```
graph TD
    Frontend --> Router
    Router --> Service
    Service --> Repository
    Repository --> SQLAlchemyORM[SQLAlchemy ORM]
    SQLAlchemyORM --> PostgreSQL
```

Shared Components

```
common/
├── exceptions.py
├── filters.py
├── helpers.py
├── pagination.py
├── response.py
├── utilities.py
└── validators.py
```

Architecture Summary

Component	Responsibility
Router	Expose REST APIs
Service	Execute business logic
Repository	Handle database operations
Models	Define database structure

Component	Responsibility
Schemas	Define request and response models
Validators	Validate data
Permissions	Control access
Dependencies	Provide reusable resources
Constants	Store reusable values
Common Package	Provide shared functionality
SQLAlchemy ORM	Manage object-relational mapping
PostgreSQL	Store application data

This architecture promotes separation of concerns, making the backend easier to maintain and extend.

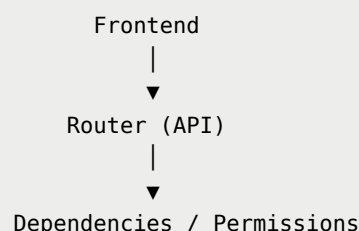
37. Conclusion

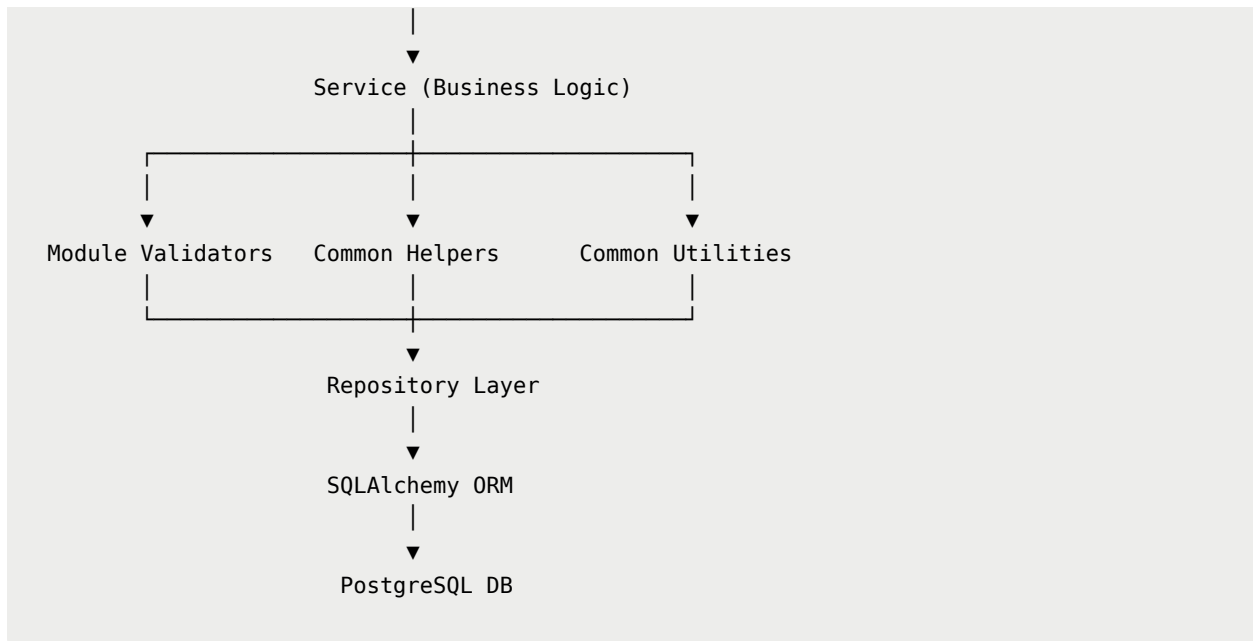
The Admitix backend is designed around a modular, layered architecture that clearly separates API handling, business logic, and database operations.

Each module follows a consistent structure, allowing new features to be developed using the same implementation pattern. Shared functionality is centralized in the `common` package, reducing code duplication and improving maintainability.

By following the Router → Service → Repository → SQLAlchemy ORM → PostgreSQL workflow, the project achieves a clean, scalable, and implementation-oriented architecture suitable for long-term development.

Complete Backend Architecture





Overall Folder Structure

```
app/
├── core/
├── common/
│   ├── exceptions.py
│   ├── filters.py
│   ├── helpers.py
│   ├── pagination.py
│   ├── response.py
│   ├── utilities.py
│   └── validators.py
├── database/
├── modules/
│   ├── admission/
│   ├── student/
│   ├── document/
│   ├── payment/
│   └── ...
├── main.py
└── requirements.txt
```